

Lecture 3/16/2025

MLP (Multi layer Perceptron) is a neural network in which neurons are "organized" in layers

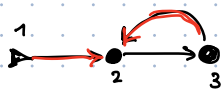


Feedforward NN: you have a dag (V, A)

(*)
$$x_i = \sigma \left(\sum_{j \in pa(i)} w_{ij} x_j \right)$$

$pa(i) = \{j \in V : j \rightarrow i \in A\}$

Ex 1



This is not a FNN because there is a cycle in the graph that defines the computational structure of the network

Eq (*) is

x_1 is given

$$x_2 = \sigma(w_{21} x_1 + w_{23} x_3)$$

Here we are in trouble because x_1 is given but we don't know the value of x_3

Backpropagation

$$L_N(w) = \frac{1}{N} \sum_{i=1}^N \ell(\underbrace{f(x_i; w)}_{\text{value}}, y_i)$$

$\ell(a, b)$

w is a vector (or tensor) of all the parameters of a NN

$$(w_{d+1,1}, w_{d+1,2}, \dots)$$

In order to use gradient-based algorithms we need to compute

$$\nabla L_N(w)$$

$$\frac{\partial L}{\partial w_{ij}} = \frac{1}{N} \sum_{k=1}^N \underbrace{l_a(f(x_k; w), y_k)}_{\text{loss}} \cdot \underbrace{\frac{\partial f(x_k; w)}{\partial w_{ij}}}_{\text{activation derivative}}$$

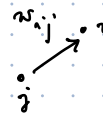
$$l(a, b) = \frac{1}{2} (a - b)^2$$

$$l_a(a, b) = a - b$$

Back prop gives you an optimal way to compute

$$\frac{\partial f}{\partial w_{ij}}(x_k; w) \quad \forall k=1, \dots, N$$

$$\frac{\partial f}{\partial w_{ij}}(x_k; w) = \frac{\partial f(x_k; w)}{\partial a_i} \frac{\partial a_i}{\partial w_{ij}} = \delta_i \frac{\partial a_i}{\partial w_{ij}}$$



a_i is called the "activation of neuron i "

$$a_i = \sum_{j \in \text{pa}(i)} w_{ij} z_j$$

Define $\delta_i = \frac{\partial f(x_k; w)}{\partial a_i}$

$$\frac{\partial a_i}{\partial w_{ij}} = \frac{\partial}{\partial w_{ij}} \left(\sum_{k \in \text{pa}(i)} w_{ik} z_k \right) = \sum_{k \in \text{pa}(i)} \delta_{ik} z_k = z_j$$

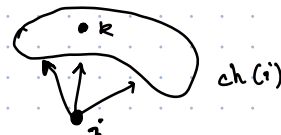
$$\frac{\partial f}{\partial w_{ij}}(x_k; w) = \boxed{\delta_i z_j} \quad \text{is } O(m)$$

$m = \# \text{ of parameters}$
or $\# \text{ of edges in the graph}$

the computation of z_j is called **Forward or Forward pass** and it is $O(m)$

How can we compute δ_i ?

$$\delta_i = \frac{\partial f}{\partial a_i}$$



$$\frac{\partial f}{\partial a_i} = \sum_{k \in \text{ch}(i)} \frac{\partial f}{\partial a_k} \frac{\partial a_k}{\partial a_i}$$

$$\text{ch}(i) = \{ j \in V : i \rightarrow j \in A \}$$

$$\frac{\partial a_k}{\partial a_i} = \frac{\partial}{\partial a_i} \left(\sum_{m \in \text{pa}(k)} w_{km} z_m \right) = \sum_{m \in \text{pa}(k)} w_{km} \delta_m$$

$$\frac{\partial a_k}{\partial a_i} = \sum_{m \in \text{pa}(k)} w_{km} \delta_m$$

$$\delta_i = \frac{\partial f}{\partial a_i} = \sum_{k \in \text{ch}(i)} \delta_k w_{ki} \sigma'(a_i) = \sigma'(a_i) \sum_{k \in \text{ch}(i)} w_{ki} \delta_k$$

$$\delta_i = r'(a_i) \sum_{k \in \text{ch}(i)} w_{ki} \delta_k$$

$$x_k = r(a_k)$$

$$\delta_k = r'(a_k)$$

The computation of δ_i is called Backward step or pass and again is $O(n)$

Convolutional Neural Networks (CNNs)

- They are an instance of FNNs.
- All the major implementation of CNNs don't use convolution

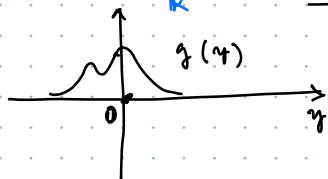
Convolution of real functions ($f: \mathbb{R} \rightarrow \mathbb{R}$)

$$f, g \in L^2(\mathbb{R}; \mathbb{R})$$

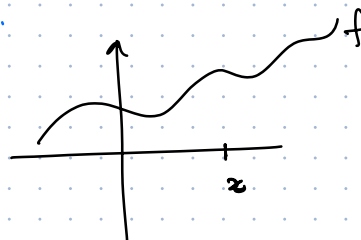
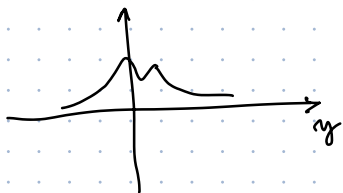
$$f, g: \mathbb{R} \rightarrow \mathbb{R}$$

$$(f * g)(x) = \int_{\mathbb{R}} f(x-y) g(y) dy$$

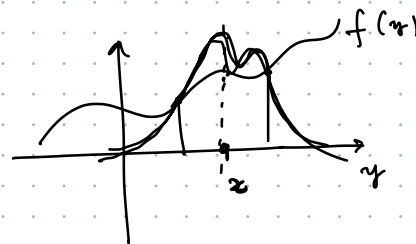
$$= \int_{\mathbb{R}} f(y) g(x-y) dy$$



$$g(-y)$$



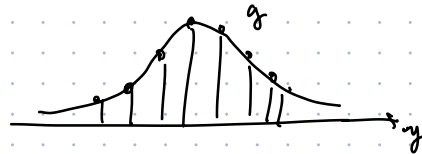
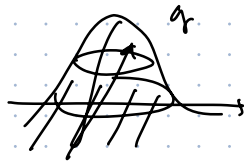
$$g(x-y)$$



Even in 2 dimension

$$(f * g)(x) = \int_{\mathbb{R}^2} f(y) g(x-y) dy$$

In NN basically g represents the weight of the Network.



0.1	2	3.4
3.1	0.2	0
0	0	0